

NAME: AMRUTHA KATTIMANI
SRN: PES1UG24CS054

SOFTWARE ENGINEERING: LAB-04
CHAT SUMMARY

REFERED PROMPT: (USED BY THE GEMINI MODEL WHICH IS INBUILT IN VSCODE)

TASK-01

I am working on a Python Pygame lab called "Skyscraper Stack Repair Lab".

I need you to work ONLY on Task 1 from the repository README.

Task 1:

Fix the inverted overlap placement bug in `game_engine.drop_block()`.

The current logic incorrectly treats `overlap <= 0` as a successful drop. The intended behavior is:

- `overlap > 0` means the active block overlaps the previous/top block and should be a successful placement.
 - `overlap <= 0` means the active block completely misses the tower and should trigger tower collapse/game over.
- Please:

1. Inspect the existing code before changing it.
2. Identify the exact existing condition responsible for the bug.
3. Make the smallest possible code change needed to fix Task 1.
4. Do not implement Tasks 2, 3, or 4 yet.
5. Do not redesign or rewrite unrelated code.
6. Preserve the existing game behavior, controls, rendering, camera logic, and restart behavior.
7. After making the change, explain exactly what was changed and why.
8. Give me a simple manual test procedure to verify both a successful overlap and a complete miss.

TASK-02

Continue from the current working repository state.

Task 1 has already been fixed and committed. Do NOT modify or undo the Task 1 fix.

Now implement ONLY Task 2 from the README:

"Perfect Placement bonus & width restoration"

Requirements:

1. In `game_engine.drop_block()`, detect when the active block is almost perfectly aligned with the top stack block.
2. Use a small alignment tolerance, approximately:
`abs(act.x - top_block.x) <= 3`
3. When this happens:
 - treat it as a perfect placement,
 - snap the active block into exact alignment,
 - preserve/restore its full width instead of trimming it,
 - display a visible golden "PERFECT!" popup label,
 - award additional bonus score points.
4. Track consecutive perfect placements.
5. After 3 perfect placements in a row, slightly expand the block width outward as specified by the task.
6. A non-perfect but valid overlap should still use the normal trimming behavior.
7. A missed placement should still trigger game over exactly as fixed in Task 1.
8. Reset the perfect streak appropriately after a non-perfect valid placement or game restart.
9. Do NOT implement Task 3 or Task 4 yet.

10. Do not rewrite unrelated parts of the project.

First inspect the current implementation and reuse its existing score, block, rendering, and game-state structures wherever possible.

After editing, explain:

- what variables/state were added,
- where perfect placement is detected,
- how the bonus works,
- how the 3-perfect streak works,
- how the popup is rendered.

Then give me manual tests for:

- A. normal overlap,
- B. perfect placement,
- C. three consecutive perfect placements,
- D. missed placement.

TASK-03

Continue from the current repository state.

Tasks 1 and 2 are already implemented and committed. Preserve them exactly.

Now implement ONLY Task 3 from the README:

"Implement falling off-cut debris animation."

Current behavior:

When an active block is trimmed because only part of it overlaps the previous block, the removed overhang disappears immediately.

Required behavior:

1. Identify the exact portion of the active block that is cut off during a normal non-perfect placement.
 2. Create a debris/off-cut object representing that removed rectangular portion.
 3. The debris must:
 - retain its correct width and height,
 - start from the correct cut-off position,
 - have gravity-based downward velocity,
 - rotate while falling,
 - be rendered visibly,
 - eventually disappear after leaving the visible game area or after an appropriate lifetime.
 4. Update the game loop/rendering so multiple debris pieces can exist and animate independently if needed.
 5. Do not break the existing tower blocks, camera scrolling, collision/overlap calculation, perfect placement system, scoring, or restart logic.
 6. Perfect placements should not create an off-cut because their width is preserved.
 7. Do NOT implement Task 4 yet.
 8. Avoid unnecessary architectural rewrites.
- Inspect the existing classes and rendering/update structure first and integrate the debris into the existing design.

After implementation, explain:

- how the cut-off rectangle is calculated,
- how debris velocity/gravity is updated,
- how rotation is handled,
- how debris is removed,
- how the camera affects debris rendering.

Then provide a manual test procedure specifically for observing the falling off-cut.

TASK-04

Continue from the current repository state.

Tasks 1, 2, and 3 are already implemented and committed. Preserve all existing behavior.

Now implement ONLY Task 4 from the README:

"Implement combo streak background color shifting."

Requirements:

1. Modify `game_engine.render()` so the background changes gradually as the tower gets higher.
 2. The background should progress through atmospheric stages such as:
 - twilight blue
 - dusk purple
 - night/starfield
 - stratosphere/near-black
 3. The transition should depend on the player's tower height / stack progress, not random changes.
 4. The transition should be visually smooth rather than abruptly switching colors.
 5. Keep the existing tower, active block, score, popup, debris, camera scrolling, and game-over rendering working.
 6. Do not remove existing UI or controls.
 7. The background progression should reset when the game restarts.
 8. Do not implement unrelated features.
 9. Avoid a complete rendering rewrite unless the existing architecture requires it.
- Inspect the existing `game_engine.py` first and determine the existing source of tower height/score/stack progress. Reuse that state rather than introducing a disconnected hardcoded counter.

After implementation, explain:

- what determines the background progression,
- how the colors are interpolated,
- what happens at different tower heights,
- how the background resets on restart.

Then provide a manual test procedure for verifying the progression.

SIMPLIFIED VERSION OF THE PROMPT FEED TO GPT AND FINAL RESPONSE SUMMARY:

Today 5:22 PM



this is the instructions and 

[SETAPESU26/54 skyscraper stack](#) this is the git which is given for me

refer the instruction pdf and the readme file from the given git repo

```
import random
import pygame
from game.block import Block
```

```
class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
```

```
self.block_height = 28
self.base_width = 180

self.font_title = pygame.font.SysFont(None, 38)
self.font_hud = pygame.font.SysFont(None, 28)
self.font_big = pygame.font.SysFont(None, 46)

self.reset()

def get_color(self, index):
    palette = [
        (230, 75, 75), # Crimson
        (240, 140, 45), # Orange
        (245, 210, 50), # Gold
        (60, 195, 110), # Green
        (50, 150, 240), # Blue
        (165, 80, 225), # Purple
    ]
    return palette[index % len(palette)]

def reset(self):
    self.score = 0
    self.game_over = False

    base_x = (self.width - self.base_width) // 2
    base_y = self.height - 60
    base_block = Block(base_x, base_y, self.base_width,
self.block_height, self.get_color(0), speed=0)
    self.stack = [base_block]

    self.spawn_active_block()

def spawn_active_block(self):
    top_block = self.stack[-1]
    next_y = top_block.y - self.block_height - 4
    speed = min(10.0, 4.5 + (len(self.stack) * 0.35))
    color = self.get_color(len(self.stack))

    start_x = 25 if random.choice([True, False]) else
self.width - 25 - top_block.width
```

```
self.active_block = Block(start_x, next_y,
top_block.width, self.block_height, color, speed=speed)

def drop_block(self):
    if self.game_over:
        return

    top_block = self.stack[-1]
    act = self.active_block

    left = max(act.x, top_block.x)
    right = min(act.x + act.width, top_block.x +
top_block.width)
    overlap = right - left

    # BUG SYMPTOM:
    # Overlap condition is inverted so hitting empty air
    succeeds while landing on the tower fails.
    is_successful_drop = overlap <= 0

    if is_successful_drop:
        trimmed_width = max(10.0, overlap)
        new_block = Block(left, act.y, trimmed_width,
self.block_height, act.color, speed=0)
        self.stack.append(new_block)
        self.score += 1

    if new_block.y < 180:
        shift_amount = self.block_height + 4
        for b in self.stack:
            b.y += shift_amount

    self.spawn_active_block()
    else:
        self.game_over = True

def handle_event(self, event):
    if self.game_over:
        if (event.type == pygame.KEYDOWN and event.key
== pygame.K_r) or \
```

```
(event.type == pygame.MOUSEBUTTONDOWN
and event.button == 1):
    self.reset()
    return

    if event.type == pygame.KEYDOWN and event.key
    == pygame.K_SPACE:
        self.drop_block()
    elif event.type == pygame.MOUSEBUTTONDOWN
    and event.button == 1:
        self.drop_block()

def update(self):
    if not self.game_over:
        self.active_block.update(self.width)

def render(self, screen):
    screen.fill((24, 27, 36))

    title_surf = self.font_title.render("Skyscraper Stack",
    True, (245, 245, 245))
    screen.blit(title_surf, (self.width // 2 -
    title_surf.get_width() // 2, 16))

    score_surf = self.font_hud.render(f"Height:
    {self.score}", True, (255, 220, 80))
    screen.blit(score_surf, (self.width // 2 -
    score_surf.get_width() // 2, 54))

    for b in self.stack:
        b.render(screen)

    if not self.game_over:
        self.active_block.render(screen)

    if self.game_over:
        overlay = pygame.Surface((self.width, self.height),
        pygame.SRCALPHA)
        overlay.fill((0, 0, 0, 195))
        screen.blit(overlay, (0, 0))
```

```

        over_surf = self.font_big.render("TOWER
        COLLAPSED!", True, (240, 75, 75))
        screen.blit(over_surf, (self.width // 2 -
        over_surf.get_width() // 2, self.height // 2 - 40))

        final_surf = self.font_hud.render(f"Final Height:
        {self.score}", True, (255, 255, 255))
        screen.blit(final_surf, (self.width // 2 -
        final_surf.get_width() // 2, self.height // 2 + 10))

        restart_surf = self.font_hud.render("Press [Space] or
        [R] to Play Again", True, (200, 200, 200))
        screen.blit(restart_surf, (self.width // 2 -
        restart_surf.get_width() // 2, self.height // 2 + 50))

```

this is the code which we need to change in order to complete the lab please go through this do not start doing anything once ur are ready i send prompt for task 1 so we can implement from then onward

Show less ^

Ready. I've gone through the lab instructions and the provided code.

—  Lab_4_VibeCoding_Student_handou...  Upgrade   ...

Send Task 1.

Sources

    ...

I ran the game and I think the block overlap logic is reversed. When the blocks actually overlap it is treating it like a miss. Can you check `drop_block()` and fix only this issue? Please don't change anything else in the game or

add the other tasks yet. so just mention what needs to be changed and why it fix the issue

</> Python



Run

```
# Change this:  
is_successful_drop = overlap <= 0  
  
# To this:  
is_successful_drop = overlap > 0
```

This fixes the issue because `overlap` is positive when the active block and the top block actually overlap. `overlap <= 0` means there is no overlap, so the original condition was reversed.



Now I need to add the perfect placement part from the README. When the new block is almost exactly aligned with the block below it, around 3 pixels difference, I want it to count as a perfect placement. It should snap properly, keep the full block width instead of cutting it, show a `PERFECT!` message and give some extra score. Can you add this to my existing code without rewriting the whole thing?

Make these **only these changes** inside `drop_block()`. The README specifies the perfect-placement threshold as `abs(act.x - top_block.x) <= 3`, with full width, snap-to-alignment, popup, bonus score, and a 3-perfect streak expansion.  [GitHub](#)

1. Add this after `is_successful_drop`

</> Python



Run

```
is_perfect = (  
    is_successful_drop
```

```
        and abs(act.x - top_block.x) <= 3
    )
```

2. Replace the current `if is_successful_drop:` block with this logic

</> Python



Run

```
if is_successful_drop:

    if is_perfect:
        new_block_x = top_block.x
        new_block_width = act.width

        self.perfect_popup_timer = 45

        if not hasattr(self, "perfect_streak"):
            self.perfect_streak = 0

        self.perfect_streak += 1

        if self.perfect_streak == 3:
            new_block_x -= 5
            new_block_width += 10

    else:
        if hasattr(self, "perfect_streak"):
            self.perfect_streak = 0

        new_block_x = left
        new_block_width = max(10.0, overlap)

    new_block = Block(
        new_block_x,
        act.y,
        new_block_width,
        self.block_height,
        act.color,
        speed=0
    )

    self.stack.append(new_block)

    self.score += 1 + (2 if is_perfect else 0)

    if new_block.y < 180:
        shift_amount = self.block_height + 4
```

```

        for b in self.stack:
            b.y += shift_amount

        self.spawn_active_block()

    else:
        self.game_over = True

```

3. Add these in `reset()`

</> Python



Run

```

self.perfect_streak = 0
self.perfect_popup_timer = 0

```

4. Add this at the beginning of `update()`

</> Python



Run

```

if self.perfect_popup_timer > 0:
    self.perfect_popup_timer -= 1

```

5. Add this in `render()` before the active block is rendered

</> Python



Run

```

if self.perfect_popup_timer > 0:
    perfect_surf = self.font_hud.render(
        "PERFECT!",
        True,
        (255, 220, 80)
    )

    screen.blit(
        perfect_surf,
        (
            self.width // 2 - perfect_surf.get_width() // 2,
            92
        )
    )

```

This keeps the existing game structure and only adds the Task 2 behavior described in the README. [GitHub](#)

</> Python



Run

```
# In reset(), add:  
self.perfect_streak = 0
```

</> Python



Run

```
# In drop_block(), inside if is_successful_drop:  
  
if is_perfect:  
    self.perfect_streak += 1  
  
    new_block_x = top_block.x  
    new_block_width = act.width  
  
    # After 3 consecutive perfect placements,  
    # slightly restore/expand the block width.  
    if self.perfect_streak == 3:  
        new_block_x -= 5  
        new_block_width += 10  
  
else:  
    # Any non-perfect placement resets the streak.  
    self.perfect_streak = 0  
  
    new_block_x = left  
    new_block_width = max(10.0, overlap)
```

The `reset()` change ensures restarting the game clears the streak, while the `else` resets it whenever the next placement is not perfect. After exactly 3 consecutive perfect placements, the block gets the width bonus.



Today 10:16 PM

I have added the overlap fix and the perfect placement feature now. Can you go through the current `game_engine.py` and make sure everything is working together properly? I want to check the overlap condition, 3 pixel perfect detection, full width on perfect placement, PERFECT popup, bonus score and the 3-perfect streak/width restoration. Don't add the debris or background feature yet. If something is wrong, fix only that.

</> Python



Run

```
# In reset()
self.perfect_streak = 0
self.perfect_popup_timer = 0
```

</> Python



Run

```
# In drop_block()

overlap = right - left

# Correct overlap condition
is_successful_drop = overlap > 0

# Perfect placement: Left edges within 3 pixels
is_perfect = (
    is_successful_drop
    and abs(act.x - top_block.x) <= 3
)

if is_successful_drop:

    if is_perfect:
        self.perfect_streak += 1

    # Snap to the block below and preserve full width
    new_block_x = top_block.x
    new_block_width = act.width
```

```

        # Show PERFECT! popup
        self.perfect_popup_timer = 45

        # Width restoration after 3 consecutive perfect placements
        if self.perfect_streak == 3:
            new_block_x -= 5
            new_block_width += 10

        else:
            # Non-perfect placement resets the streak
            self.perfect_streak = 0

            new_block_x = left
            new_block_width = max(10.0, overlap)

        new_block = Block(
            new_block_x,
            act.y,
            new_block_width,
            self.block_height,
            act.color,
            speed=0
        )

        self.stack.append(new_block)

        # Normal = +1, perfect = +3
        self.score += 1 + (2 if is_perfect else 0)

        if new_block.y < 180:
            shift_amount = self.block_height + 4

            for b in self.stack:
                b.y += shift_amount

        self.spawn_active_block()

    else:
        self.game_over = True

```

</> Python



Run

In update()

```

if self.perfect_popup_timer > 0:
    self.perfect_popup_timer -= 1

```

</> Python



Run

In render(), before rendering the active block

```

if self.perfect_popup_timer > 0:
    perfect_surf = self.font_hud.render(
        "PERFECT!",
        True,
        (255, 220, 80)
    )

    screen.blit(
        perfect_surf,
        (
            self.width // 2 - perfect_surf.get_width() // 2,
            92
        )
    )

```



Pasted text(20260930-16...
Document

so one final check is this code overall functionally correct when we consider for all the 4 task

Yes. Based on the current `game_engine.py` you uploaded and the repository README, all 4 tasks are functionally implemented. Pasted text

- **Task 1 — Overlap fix:** Correct: `overlap > 0` succeeds and `<= 0` causes game over.



- **Task 2 — Perfect placement:** Correct: ≤ 3 px alignment, snap to the previous block, preserve full width, `PERFECT!` popup, +3 score, and 3-perfect width expansion.



- **Task 3 — Off-cut debris:** Correct: non-perfect trimming creates left/right `OffcutDebris`, updates it, removes finished debris, scrolls it with the tower, and renders it. Pasted text

- **Task 4 — Background:** Correct: `render_background()` replaces the flat background with progressive sky gradients and adds stars at higher stack levels, matching the README requirement.  Pasted text

One small functional observation: the width bonus triggers when the streak reaches exactly 3 (`== 3`). That's consistent with the task wording "If the player lands 3 perfect placements in a row, slightly expand the block width."  GitHub

So I would not make any further code changes to this `game_engine.py` for the four README tasks.

Sources



ChatGPT can make mistakes. Check important info.